```
  describe "shouldSatisfy" $ do
    it "should satisfy the condition that 10 is greater than
5" $ do
      10 `shouldSatisfy` (> 5)

  describe "shouldThrow" $ do
    it "should throw an exception when taking head of an
empty list" $ do
      evaluate (1 `div` 0) `shouldThrow` anyException
```

Executing the above code provides the following output:

```
ghci> main

shouldReturn
  should successfully return an Integer
shouldSatisfy
  should satisfy the condition that 10 is greater than 5
shouldThrow
  should throw an exception when taking head of an empty list

Finished in 0.0015 seconds
3 examples, 0 failures
```

The *evaluate* function can be used to check for exceptions. Its type signature is as follows:

```
ghci> :t evaluate
evaluate :: a -> IO a
```

You can also re-use HUnit tests and integrate them with Hspec. An example is shown below:

```
import Test.HUnit
import Test.Hspec
import Test.Hspec.Contrib.HUnit (fromHUnitTest)

test1 = TestList [ TestLabel "Test subtraction" foo ]

foo :: Test
foo = TestCase $ do
  3 @?= (4 - 1)

main = hspec $ do
  describe "Testing equality" $ do
    it "returns 3 for the sum of 2 and 1" $ do
      3 `shouldBe` (2 + 1)

  describe "Testing subtraction" $ do
    fromHUnitTest test1
```

Testing the code in GHCi produces the following output:

```
ghci> main
Loading package hspec-contrib-0.2.0 ... linking ... done.

Testing equality
  returns 3 for the sum of 2 and 1
Testing subtraction
  Test subtraction

Finished in 0.0004 seconds
2 examples, 0 failures
```

The Hspec tests can also be executed in parallel. For example, computing the Fibonacci for a set of numbers and asserting their expected values can be run in parallel, as shown below:

```
import Test.Hspec

fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)

main = hspec $ parallel $ do
  describe "Testing Fibonacci" $ do
    it "must return 6765 for fib 20" $ do
      fib 20 `shouldBe` 6765

    it "must return 75025 for fib 25" $ do
      fib 25 `shouldBe` 75025

    it "must return 832040 for fib 30" $ do
      fib 30 `shouldBe` 832040

    it "must return 9227465 for fib 35" $ do
      fib 35 `shouldBe` 9227465
```

You can compile and execute the above code as shown below:

```
$ ghc -threaded parallel.hs
Linking parallel ...

$ ./parallel +RTS -N -RTS

Testing Fibonacci
  must return 6765 for fib 20
  must return 75025 for fib 25
  must return 832040 for fib 30
  must return 9227465 for fib 35

Finished in 0.9338 seconds
4 examples, 0 failures
```

END

By: Shakthi Kannan

The author is a free software enthusiast and blogs at *shakthimaan.com*